

OBJETIVOS

- I/O: teclado.
- Tratamento de interrupção
- Criação de um *shell* muito simples.

TAREFA 4: CONSTRUÇÃO DE UM SHELL SIMPLES (de 06/02/2025 a 09/02/2025)

Para a criação do *shell*, esta tarefa utiliza as últimas modificações nas bibliotecas desenvolvidas até o momento e introduz novas modificações. Para esta prática, iremos criar um arquivo de biblioteca e sua contraparte *shell.h* e *shell.c*. Além disso, faça as seguintes modificações indicadas no Código 11 em *kernel.c*. Por ser muito simples, o arquivo de biblioteca para *shell.h* fica a cargo do aluno. O arquivo para *shell.c* é apresentado no Código 12. Perceba neste código que introduzimos a função `malloc()`. Esta será inserida no contexto do arquivo *util.h* e *util.c*. Além disso, também foram inseridas modificações em funções pertencentes a *screen.h* e *screen.c*. (Código 13). Fica a cargo do aluno introduzir todas as modificações necessárias nos arquivos “*.h” referentes às funções introduzidas nas bibliotecas, conforme apresentado. Além disso, também será necessário fazer alguma modificação no arquivo *Makefile*, de modo a implementar o suporte a *shell.h* e *shell.c* (Código 14).

```
//...
#include "../include/shell.h"

int kmain()
{
    isr_install();
    clearScreen();

    //...

    print("Bem vindo ao B.OS ----->\n\n\n\n\nEntre um
comando\n");

    launch_shell(0);
}
```

Código 11: Modificações a serem inseridas em *kernel.c*.

```

#include "../include/util.h"

// ...

/* NOVAS funcoes */

string int_to_string(int n) {
    string ch = malloc(50);
    int_to_ascii(n, ch);
    int len = strlen(ch);
    int i = 0,
        j = len - 1;
    while (i < (len/2 + len%2)) {
        char tmp = ch[i];
        ch[i] = ch[j];
        ch[j] = tmp;
        i++;
        j--;
    }
    return ch;
}

int str_to_int(string ch) {
    int n = 0;
    int p = 1;
    int strlen = strlen(ch);
    int i;

    for(i = strlen-1; i >= 0; i--) {
        n += ((int)(ch[i] - '0')) * p;
        p *= 10;
    }
    return n;
}

void * malloc(int nbytes) { // maneira simples mas
                             // malloc - nao faz uso de
                             // mcb

    char variable[nbytes];
    return &variable;
}

```

Código 12: Modificações a serem inseridas em *util.c*.

Algumas observações são válidas: o processamento do *backspace* deverá ser adequado de modo a lidar com o “*prompt*” do *shell*. Se o usuário começar a dar *backspace* indiscriminadamente, acabará apagando o *prompt*. Além disso, ao digitar um comando, se o *backspace* for pressionado, isso irá influenciar no tratamento do comando (ex. digitando-se *clear*, pressionando-se o *backspace* e novamente o ‘r’, não resultará no *clear*). Assim sendo, tratem esse *caveat*.

```

#include "../include/screen.h"

// ...

int color = 0x0F; // NOVO: armazena a cor corrente
                  // usada no terminal

// NOVO: funcao clearLine() modificada para usar color
void clearLine(uint8 from, uint8 to)
{
    //...

    for (i; i < (sw * to * sd); i++) // NOVO
    {
        vidmem[(i/2)*2+1] = color; // NOVO: referente a cor
        vidmem[(i/2)*2] = 0; // NOVO: referente ao caractere
    }
}

// ...

/* NOVAS funcoes */
void set_screen_color(int text_color, int bg_color) {
    color = (bg_color << 4) | text_color;
}

void set_screen_color_from_color_code(int color_code) {
    color = color_code;
}

void print_colored(string ch, int text_color, int
bg_color) {
    int current_color = color;
    set_screen_color(text_color, bg_color);
    print(ch);
    set_screen_color_from_color_code(current_color);
}

```

Código 13: Modificações a serem inseridas em *screen.c*.

```

//...
#include "../include/shell.h"

void launch_shell(int n) {
    string ch = (string) malloc(200); //definido em util.h
    do {
        print("B.OS ");
        print(int_to_string(n));
        print(">");
        ch = readStr(); // memory_copy(readStr(), ch, 100);
        if(strEq1(ch, "cmd")) {
            print("\nVoce ja esta no cmd. Novo shell recursivo
aberto\n");
            launch_shell(n+1);
        }
        else if (strEq1(ch, "clear"))
            clearScreen();
        else if (strEq1(ch, "soma"))
            sum();
        else if (strEq1(ch, "exit"))
            print("\nAte logo!\n");
        else if (strEq1(ch, "echo"))
            echo();
        else if (strEq1(ch, "sort"))
            sort();
        else if (strEq1(ch, "fibo"))
            fibonacci();
        else if (strEq1(ch, "mmc"))
            gcd();
        else if (strEq1(ch, "help"))
            help();
        else if (strEq1(ch, "cor"))
            set_bg_color();
        else {
            print("\nComando desconhecido\n");
        }
    } while(!strEq1(ch, "exit"));
}

void sum() {
    int n, i, s;

    print("\nQuantos n.os: ");
    n = str_to_int(readStr());
    i = 0;
    print("\n");
    int arr[n];
    fill_array(arr, n);
    s = sum_array(arr, n);
    print("Resultado: ");
    print(int_to_string(s));
    print("\n");
}

void fill_array(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        print("ARR[");
        print(int_to_string(i));
        print("]:");
        arr[i] = str_to_int(readStr());
        print("\n");
    }
}

```

```

void print_array(int arr[], int n) {
    int i;

    for (i = 0; i < n; i++) {
        print(int_to_string(arr[i]));
        print(" ");
    }
    print("\n");
}

int sum_array(int arr[], int n) {
    int i;
    int s = 0;

    for (i = 0; i < n; i++) {
        s += arr[i];
    }
    return s;
}

void echo() {
    print("\n");
    string str = readStr();
    print("\n");
    print(str);
    print("\n");
}

void sort() {
    int arr[100];
    print("\nTamanho do vetor: ");
    int n = str_to_int(readStr());
    print("\n");
    fill_array(arr, n);
    print("Antes da ordenacao:\n");
    print_array(arr, n);
    print("\nOrdem: (1 - crescente/ 0 - decrescente): ");
    int order = str_to_int(readStr());
    insertion_sort(arr, n, order);
    print("Apos ordenacao:\n");
    print_array(arr, n);
}

void insertion_sort(int arr[], int n, int order) { // 1 -
crescente e 0 - decrescente
    int i;

    for (i = 0; i < n; i++) {
        int aux = arr[i];
        int j = i;

        while((j > 0) && ((aux < arr[j-1]) && order)) {
            arr[j] = arr[j-1];
            j = j-1;
        }
        arr[j] = aux;
    }
}

void fibonacci() {
    int i, n;

```

```

    print("\nQuantos elementos: ");
    n = str_to_int(readStr());
    print("\n");
    for (i = 0; i < n; i++) {
        print("Fibo ");
        print(int_to_string(i));
        print(" : ");
        print(int_to_string(fibo(i)));
        print("\n");
    }
}

int fibo(int n) {
    if (n < 2)
        return 1;
    else
        return fibo(n-1) + fibo (n-2);
}

void set_bg_color() {
    print("\nCodigos de cores: ");
    print("\n0 : preto");
    print_colored("\n1 : azul", 1,0);
    print_colored("\n2 : verde", 2,0);
    print_colored("\n3 : ciano", 3,0);
    print_colored("\n4 : vermelho", 4,0);
    print_colored("\n5 : purpura", 5,0);
    print_colored("\n6 : laranja", 6,0);
    print_colored("\n7 : cinza", 7,0);
    print_colored("\n8 : cinza escuro", 8,0);
    print_colored("\n9 : azul claro", 9,0);
    print_colored("\n10 : verde claro", 10,0);
    print_colored("\n11 : azul mais claro", 11,0);
    print_colored("\n12 : vermelho claro", 12,0);
    print_colored("\n13 : rosa", 13,0);
    print_colored("\n14 : amarelo", 14,0);
    print_colored("\n15 : branco", 15,0);

    print("\n\n Cor do texto: ");
    int text_color = str_to_int(readStr());
    print (" \n\n Cor do fundo: ");
    int bg_color = str_to_int(readStr());
    set_screen_color(text_color, bg_color);
    clearScreen();
}

void help() {
    print("\ncmd\t: executa um novo shell recursivo");
    print("\nclear\t: limpa a tela");
    print("\nsoma\t: computa a soma de n numeros");
    print("\necho\t: imprime um dado texto");
    print("\nsort\t: ordena n numeros");
    print("\nfibo\t: imprime os n primeiros numeros da
serie de Fibonacci");
    print("\ncor\t: muda a cor do terminal");
    print("\nquit\t: sai do shell corrente");

    print("\n\n");
}

```

Código 14: Implementação de *shell.c*.

Entregue no SIGAA todos os códigos e scripts resultantes desta tarefa para análise.